

QuitQOS — API Tasarımı

v0.2 — MVP Backend: Spring Boot 4.1 · Java 21 · PostgreSQL · Spring Security (JWT) · Firebase Auth + FCM

Bu doküman [project brief v0.4](#), [ERD](#) ve [user journeys](#) baz alınarak hazırlanmıştır.

1. Genel Prensipler

Konu	Karar
Base URL	/api/v1
Format	JSON (request & response)
Kimlik	Authorization: Bearer <JWT>
ID tipi	UUID (string)
Zaman	ISO-8601, UTC (2026-06-24T18:30:00Z)
Dil	İçerik (milestone başlık/açıklama) sunucudan TR döner
Versiyonlama	URL path (/v1)

Standart hata formatı

```
{
  "timestamp": "2026-06-24T18:30:00Z",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "QuitAttempt bulunamadı",
  "path": "/api/v1/quit-attempts/abc"
}
```

Kod	Anlam
400	Validation / malformed request
401	Token yok / geçersiz
403	Yetki yok (örn. guest leaderboard'a giremez)
404	Kaynak bulunamadı
409	Çakışma (örn. zaten aktif bir streak var)
422	İş kuralı ihlali (örn. backdated tarih gelecekte)

2. Kimlik & Kullanıcı Tipleri

İki kullanıcı tipi var (bkz. brief):

- **Guest** — kayıt yok, veriler **cihazda** tutulur. Backend'e gerek duymadan tüm temel özellikler çalışır. Leaderboard'a giremez.
- **Kayıtlı** — Google/Apple ile Firebase Auth. Veriler sunucuda senkronize edilir, leaderboard'a katılır.

Token akışı

1. Mobil uygulama Firebase ile login olur → **Firestore ID Token** alır.
2. Bu token backend'e gönderilir, backend doğrular (Firestore Admin SDK) → kendi **app JWT** (access) + **refresh token**'ını üretir.
3. Sonraki tüm isteklerde access JWT kullanılır; süresi dolunca refresh ile sessizce yenilenir.

Oturum / token stratejisi (karar)

Mobil UX hedefi: **kullanıcı bir kez login olur, logout'a kadar bir daha login ekranı görmez.**

- **Access JWT** — kısa ömürlü (~1 saat), stateless, her istekte taşınır.
- **Refresh token** — **DB'de saklanır**, uzun ömürlü (~180 gün), **her kullanımda rotate** edilir. Aktif kullanan kullanıcı pratikte hiç süre dolumu yaşamaz.
- **Logout** — DB'deki refresh satırı silinir → oturum sunucu tarafında gerçekten sonlanır. (Saf stateless'ın aksine token gerçekten iptal edilebilir.)

Guest tasarım notu (karar)

Guest tamamen **device-local** çalışır: quit attempt, sayaç, ulaşılan milestone ve rozetler **cihazda** saklanır (AsyncStorage/SQLite).

Backend'e **hiç** istek gitmez; sayaç ve milestone tespiti `startedAt` 'tan client-side hesaplanır.

- **Bildirimler:** Guest sunucuda olmadığı için milestone bildirimleri **cihazda local notification** olarak kurulur. Milestone offset'leri sabit olduğundan, streak başlarken 9 bildirimin tümü önceden zamanlanır (server FCM gerekmez).
- **Upgrade:** Guest → kayıtlı geçişte cihazdaki veriler `POST /users/me/sync` ile sunucuya bir kez merge edilir (bkz. §8).
- Backend yalnızca **kayıtlı** kullanıcıları tutar; her kurulum DB'de hesap yaratmaz.

3. Auth Endpoints

`POST /api/v1/auth/firebase`

Firestore ID token ile login/register. Kullanıcı yoksa oluşturur (upsert), app JWT döner.

Auth: Public

```
// Request
{
  "firebaseIdToken": "<firebase-id-token>",
  "displayName": "Berkan",          // ilk kayıta opsiyonel override
  "avatarUrl": "https://...",      // opsiyonel
  "fcmToken": "<device-fcm-token>" // opsiyonel, login anında set edilebilir
}
```

```
// 200 OK
{
  "accessToken": "<app-jwt>",
  "refreshToken": "<refresh-token>",
  "expiresIn": 3600,
  "user": {
    "id": "9f1c...",
    "isGuest": false,
    "displayName": "Berkan",
    "avatarUrl": "https://...",
    "notificationsEnabled": true,
    "createdAt": "2026-06-24T18:30:00Z"
  }
}
```

POST /api/v1/auth/refresh

Access JWT yenileme. Refresh token rotate edilir (eski geçersizleşir, yeni döner).

```
// Request { "refreshToken": "<refresh-token>" }
// 200 OK { "accessToken": "<app-jwt>", "refreshToken": "<new-refresh-token>", "expiresIn": 3600 }
// 401 refresh token geçersiz / iptal edilmiş (logout sonrası)
```

POST /api/v1/auth/logout

Refresh token'ı DB'den siler → oturum sunucu tarafında sonlanır.

```
// Request { "refreshToken": "<refresh-token>" }
// 204 No Content
```

4. User Endpoints

GET /api/v1/users/me

Profil bilgisi. **Auth:** required

```
// 200 OK
{
  "id": "9f1c...",
  "isGuest": false,
  "displayName": "Berkan",
  "avatarUrl": "https://...",
  "notificationsEnabled": true,
  "createdAt": "2026-06-24T18:30:00Z",
  "updatedAt": "2026-06-24T18:30:00Z"
}
```

PATCH /api/v1/users/me

Profil + bildirim tercihi güncelle. Tüm alanlar opsiyonel (partial update).

```
// Request { "displayName": "Berkant T.", "avatarUrl": "https://...", "notificationsEnabled": false }  
// 200 OK -> güncel user objesi
```

notificationsEnabled=false iken sunucu o kullanıcıya milestone push'u **göndermez** (kayıt yine de UserMilestone'a düşer).

PUT /api/v1/users/me/fcm-token

Push bildirimleri için cihaz token'ını kaydet/güncelle.

```
// Request { "fcmToken": "<device-fcm-token>" }  
// 204 No Content
```

5. Quit Attempt Endpoints (Bırakma Sayacı)

Bir kullanıcının aynı anda **en fazla bir aktif** (status=ACTIVE) quit attempt'i olabilir.

POST /api/v1/quit-attempts

Yeni streak başlat. Backdated başlangıç desteklenir (User Story: "Şimdi bırakıyorum" + "geçmiş tarih seçilebilir").

Auth: required

```
// Request  
{  
  "startedAt": "2026-06-20T09:00:00Z" // opsiyonel; yoksa "şimdi"  
}
```

- startedAt verilmez → now() , isBackdated=false .
- startedAt geçmişte → isBackdated=true .
- startedAt gelecekte → **422**.
- Zaten aktif attempt varsa → **409**.

```
// 201 Created  
{  
  "id": "a12...",  
  "startedAt": "2026-06-20T09:00:00Z",  
  "endedAt": null,  
  "status": "ACTIVE",  
  "isBackdated": true,  
  "elapsed": { "days": 4, "hours": 9, "minutes": 30, "seconds": 12 }  
}
```

GET /api/v1/quit-attempts/current

Aktif streak + canlı geçen süre. Anasayfa sayacı bunu kullanır.

```
// 200 OK -> tek attempt objesi (yukarıdaki gibi) | 404 aktif yoksa
```

GET /api/v1/quit-attempts

Geçmiş tüm denemeler. Journey "List streak stats → Sort time based".

Query: ?sort=startedAt,desc (default), ?status=RELAPSED

```
// 200 OK
{
  "items": [
    { "id": "a12...", "startedAt": "...", "endedAt": "...", "status": "RELAPSED", "durationSeconds": 388212 },
  ],
  "longestStreakSeconds": 1209600
}
```

GET /api/v1/quit-attempts/{id}

Tek deneme detayı (+ kazanılan milestone'lar).

POST /api/v1/quit-attempts/{id}/relapse

Relapse akışı (User Story: "Maalesef içtim"). Aktif attempt'i kapatır: `status=RELAPSED` , `endedAt=now()` . Geçmişte saklanır.

Auth: required

```
// Request (opsiyonel) { "endedAt": "2026-06-24T18:00:00Z" }
// 200 OK
{
  "id": "a12...",
  "status": "RELAPSED",
  "endedAt": "2026-06-24T18:00:00Z",
  "durationSeconds": 388212,
  "lostBadges": 3 // rozet kaybı uyarısı için
}
```

Yeni streak başlatmak ayrı bir `POST /quit-attempts` çağrısıdır (frontend relapse onayından sonra tetikler).

6. Milestone & Health Endpoints

Milestone statik referans verisidir (seed): `offsetMinutes` , `title` , `description` , `badgeId` . Brief'teki tablo: 20dk, 8sa, 24sa, 48sa, 72sa, 1hf, 1ay, 3ay, 1yıl.

GET /api/v1/milestones

Tüm milestone tanımları (sağlık istatistikleri ekranı + ilerleme barı için).

```
// 200 OK
{
  "items": [
    {
      "id": "m1",
      "offsetMinutes": 20,
      "title": "20 dakika",
      "description": "Nabız ve tansiyon normale dönmeye başlar.",
      "badge": { "id": "b1", "name": "İlk Adım", "iconUrl": "https://..." }
    }
  ]
}
```

GET /api/v1/quit-attempts/{id}/milestones

Bir denemede ulaşılan + bekleyen milestone'lar (achievedAt ile).

```
// 200 OK
{
  "items": [
    { "milestoneId": "m1", "offsetMinutes": 20, "achieved": true, "achievedAt": "2026-06-20T09:20:00Z" },
    { "milestoneId": "m2", "offsetMinutes": 480, "achieved": false, "etaAt": "2026-06-20T17:00:00Z" }
  ]
}
```

Sağlık istatistikleri ayrı bir entity gerektirmez: aktif attempt'in geçen süresi milestone tablosuyla eşleştirilerek "şu an vücudunda ne oluyor" türetilir.

7. Badge & Achievement Endpoints (Rozet Koleksiyonu)

GET /api/v1/users/me/achievements

Journey "List Achievement → earned badges + health benefits". Kullanıcının UserMilestone kayıtlarından kazanılan rozetler.

```
// 200 OK
{
  "earnedBadges": [
    { "badgeId": "b1", "name": "İlk Adım", "iconUrl": "https://...", "achievedAt": "2026-06-20T09:20:00Z"
  },
  "healthBenefits": [
    { "milestoneId": "m1", "title": "20 dakika", "description": "Nabız ve tansiyon normale dönmeye başlar"
  },
  "totalBadges": 9,
  "earnedCount": 1
}
```

GET /api/v1/badges

Tüm rozet kataloğu (kazanılmamışlar kilitli gösterilebilir).

8. Guest → Kayıtlı Geçiş (Sync)

Journey "Guest → Registered Upgrade: Merge guest data / Sync local data".

POST /api/v1/users/me/sync

Cihazdaki guest verisini (quit attempt geçmişi) sunucuya yükler/merge eder. Login sonrası bir kez çağrılır.

Auth: required (kayıtlı kullanıcı)

```
// Request
{
  "quitAttempts": [
    {
      "startedAt": "2026-05-01T08:00:00Z",
      "endedAt": "2026-05-10T08:00:00Z",
      "status": "RELAPSED",
      "isBackdated": false,
      "localId": "device-uuid-1" // idempotency / çift kayıt önleme
    }
  ]
}
```

```
// 200 OK
{
  "merged": 1,
  "skipped": 0,
  "currentAttemptId": "a99..."
}
```

Merge stratejisi: `localId` ile daha önce sync edilmiş kayıtlar atlanır (idempotent). Çakışan aktif streak'lerde **en erken startedAt** kazanır.

9. Leaderboard Endpoints

Sadece **kayıtlı** kullanıcılar. Guest → **403** (frontend register prompt gösterir).

GET `/api/v1/leaderboard`

Streak sıralaması. **Default metrik:** **current** (şu an devam eden en uzun aktif streak — canlı rekabet; relapse edince sıralamadan düşersin). **longest** (tüm zamanların rekoru) de desteklenir.

Query: `?metric=current|longest` (default `current`), `?limit=50`, `?cursor=...`

```
// 200 OK
{
  "metric": "current",
  "items": [
    { "rank": 1, "userId": "u1", "displayName": "Ayşe", "avatarUrl": "...", "streakSeconds": 2592000 }
  ]
}
```

GET `/api/v1/leaderboard/me`

Kullanıcının kendi sırası.

```
// 200 OK { "rank": 42, "streakSeconds": 388212, "metric": "current" }
// 403    guest kullanıcı
```

10. Bildirimler (Server-side)

Milestone bildirimleri **client endpoint'i değildir**; sunucu tarafında zamanlanmış iş ile tetiklenir:

- Bir **scheduler** (örn. her dakika) aktif attempt'leri tarar.
- `elapsed >= offsetMinutes` olan ve henüz `UserMilestone` kaydı olmayan milestone'lar için:
 1. `UserMilestone` kaydı oluştur (`achievedAt`),
 2. Rozet ata,
 3. FCM push gönder, `notificationSentAt` set et.
- Push payload milestone başlığı + bilimsel mesaj içerir; tıklanınca ilgili sağlık ekranına derin link (deep link) açılır.
- `User.notificationsEnabled=false` olan kullanıcılara push gönderilmez (`UserMilestone` kaydı yine oluşur).

Guest bildirimleri: Guest sunucuda olmadığından bu scheduler onları görmez; guest push'ları cihazda **local notification** olarak streak başlarken önceden zamanlanır (bkz. §2).

Bildirim tercihleri (brief: "ayarlardan yönetebilir") MVP'de **tek `notificationsEnabled` flag** ile tutulur (`PATCH /users/me`).

Granüler tercihler (milestone bazlı, sessiz saat) v2'ye bırakılmıştır.

11. Endpoint Özet Tablosu

Method	Path	Auth	Açıklama
POST	/auth/firebase	public	Firestore login/register → JWT + refresh
POST	/auth/refresh	public	Access JWT yenile (refresh rotate)
POST	/auth/logout	public	Refresh token'ı iptal et
GET	/users/me	user	Profil
PATCH	/users/me	user	Profil güncelle
PUT	/users/me/fcm-token	user	FCM token kaydet
POST	/users/me/sync	user	Guest verisini merge et
GET	/users/me/achievements	user	Kazanan rozet + sağlık faydaları
POST	/quit-attempts	user	Streak başlat (backdated destekli)
GET	/quit-attempts/current	user	Aktif streak + canlı sayaç
GET	/quit-attempts	user	Geçmiş denemeler
GET	/quit-attempts/{id}	user	Deneme detayı
POST	/quit-attempts/{id}/relapse	user	Relapse, sayaç sıfırla
GET	/quit-attempts/{id}/milestones	user	Deneme milestone ilerlemesi
GET	/milestones	user	Milestone kataloğu
GET	/badges	user	Rozet kataloğu
GET	/leaderboard	user (kayıtlı)	Streak sıralaması
GET	/leaderboard/me	user (kayıtlı)	Kendi sıran

12. Kararlar (v0.2'de kapatıldı)

- ✓ **Refresh token** → DB'de saklanan, ~180 gün ömürlü, her kullanımda rotate edilen refresh token. Access JWT ~1 saat. Logout DB satırını siler. Hedef: kullanıcı bir kez login olur, logout'a kadar bir daha login görmez. (bkz. §2, §3)
- ✓ **Guest backend kaydı** → Guest tamamen device-local; backend'e hiç dokunmaz. Push'lar cihazda local notification. Upgrade'de POST /users/me/sync ile bir kez merge. Backend yalnızca kayıtlı kullanıcıyı tutar. (bkz. §2, §8)
- ✓ **Leaderboard metric** → Default **current** (aktif streak, canlı rekabet). **longest** opsiyonel query ile sunulur. (bkz. §9)
- ✓ **Bildirim tercihleri** → User'a tek **notificationsEnabled** boolean. Granüler tercih v2. (bkz. §4, §10)

Seed verisi (referans)

ERD'deki Milestone ve Badge tabloları statiktir; uygulama açılışında seed edilir. `offsetMinutes` değerleri:

Milestone	offsetMinutes
20 dakika	20
8 saat	480
24 saat	1.440
48 saat	2.880
72 saat	4.320
1 hafta	10.080
1 ay	43.200
3 ay	129.600
1 yıl	525.600